

Embedded Linux Kernel Optimization

with Boot Time Reduction and Performance Tuning

Project Report

Author	Praveen Manne
Project type	Linux boot analysis and service tuning
Tools	systemd-analyze, systemctl, Bash, nano
Environment	Ubuntu Linux under WSL (systemd enabled)
Result	Userspace startup 22.287 s to 7.527 s
Repository	github.com/praveenmanne09

Contents

- 1. Abstract
- 2. Objectives
- 3. Background Theory
- 4. Test Environment and Methodology
- 5. Baseline Measurement (Before Optimization)
- 6. Optimization Script
- 7. Measurement After Optimization
- 8. Results and Comparison
- 9. Discussion
- 10. Advantages, Limitations, Conclusion, Future Scope, References

1. Abstract

Embedded Linux is used in IoT devices, consumer electronics, industrial controllers and communication systems, where fast boot time and efficient performance matter. This project studies the Linux boot process and applies a practical, repeatable optimization method: measure the baseline boot time, identify slow units with `systemd-analyze blame`, disable non-essential services with a Bash script, reboot, and measure again. In the test run the userspace startup time dropped from 22.287 s to 7.527 s.

2. Objectives

- Understand the Embedded Linux boot flow (bootloader, kernel, root filesystem, init system, services)
- Measure the baseline boot time and identify the slowest startup units
- Disable unnecessary services using an automated shell script
- Measure the boot time again and compare the results before and after optimization

3. Background Theory

Boot time is the time from power-on until the system is ready to use. A typical Embedded Linux boot has these stages:

#	Stage	Role
1	Boot ROM	Starts the initial boot process
2	Bootloader (e.g. U-Boot)	Loads the Linux kernel into memory
3	Kernel initialization	Initializes hardware and drivers
4	Root filesystem mount	Provides system files
5	Init system (systemd)	Starts background services
6	Services start	Networking, logging, device management and more
7	Application ready	System is usable

Boot time grows because of bootloader delay, too many kernel drivers, debug options, slow filesystem mounts, too many startup services and heavy applications launched automatically. This project targets the **startup services** stage, which can be tuned without rebuilding the kernel.

4. Test Environment and Methodology

The tests were run in a Ubuntu terminal (user manne, host DESKTOP-QAJ0UBH) on Windows Subsystem for Linux with systemd. The method follows eight steps:

Step	Action
1	Measure the current boot time (<code>systemd-analyze</code>)
2	Identify slow services (<code>systemd-analyze blame head -10</code>)
3	Create the optimization script (<code>nano optimize_services.sh</code>)
4	Make it executable (<code>chmod +x optimize_services.sh</code>)

Step	Action
5	Run it (./optimize_services.sh)
6	Reboot the system
7	Measure boot time and slow services again
8	Compare the before and after results

5. Baseline Measurement (Before Optimization)

Command: `systemd-analyze` — reports the total startup time. On WSL only the userspace phase is reported, because the firmware and kernel phases are managed by the Windows host.

```
manne@DESKTOP-QAJOUBH:~$ systemd-analyze
Startup finished in 22.287s (userspace)
graphical.target reached after 21.861s in userspace.
```

Figure 1: `systemd-analyze` before optimization - startup finished in 22.287 s

Command: `systemd-analyze blame | head -10` — lists the ten slowest units.

```
manne@DESKTOP-QAJOUBH:~$ systemd-analyze blame | head -10
4.242s dev-sdd.device
3.841s landscape-client.service
3.493s snapd.seeded.service
2.846s wsl-pro.service
2.103s snapd.service
1.884s systemd-resolved.service
1.817s rsyslog.service
1.380s systemd-logind.service
1.320s systemd-timesyncd.service
930ms e2scrub_reap.service
```

Figure 2: ten slowest units before optimization

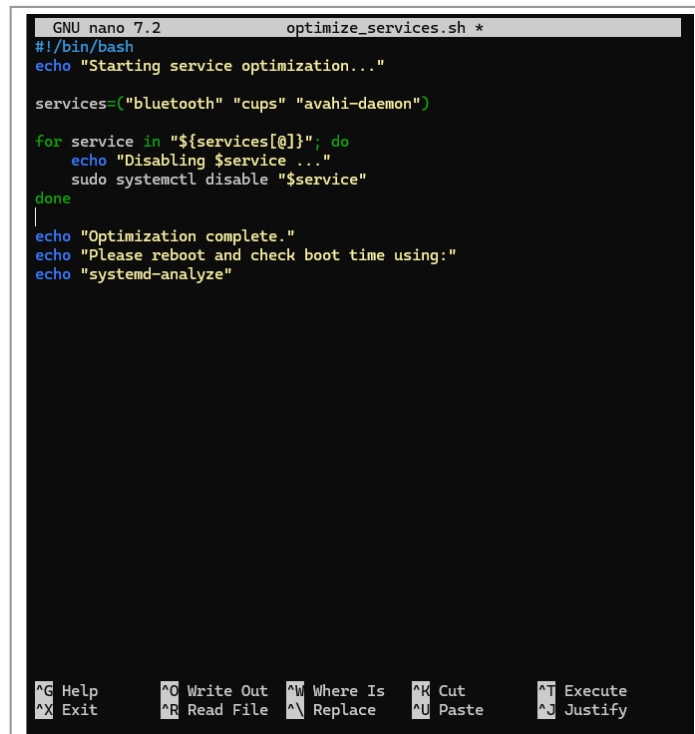
Rank	Service / Unit	Time taken
1	dev-sdd.device	4.242 s
2	landscape-client.service	3.841 s
3	snapd.seeded.service	3.493 s
4	wsl-pro.service	2.846 s
5	snapd.service	2.103 s
6	systemd-resolved.service	1.884 s
7	rsyslog.service	1.817 s
8	systemd-logind.service	1.380 s
9	systemd-timesyncd.service	1.320 s
10	e2scrub_reap.service	930 ms

6. Optimization Script

The script `optimize_services.sh` disables three commonly unneeded services: **bluetooth** (wireless support), **cups** (printing) and **avahi-daemon** (network service discovery). A Bash array holds the service names and a loop disables each one.



Figure 3: creating the script with nano



```
GNU nano 7.2      optimize_services.sh *
#!/bin/bash
echo "Starting service optimization..."

services=("bluetooth" "cups" "avahi-daemon")

for service in "${services[@]}; do
    echo "Disabling $service ..."
    sudo systemctl disable "$service"
done
echo "Optimization complete."
echo "Please reboot and check boot time using:"
echo "systemd-analyze"
```

Figure 4: contents of `optimize_services.sh` in the nano editor

```
#!/bin/bash
echo "Starting service optimization..."

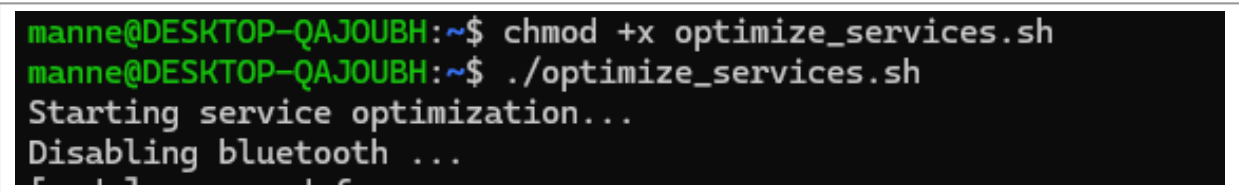
services=("bluetooth" "cups" "avahi-daemon")

for service in "${services[@]}; do
    echo "Disabling $service ..."
    sudo systemctl disable "$service"
done

echo "Optimization complete."
echo "Please reboot and check boot time using:"
echo "systemd-analyze"
```

Note on syntax: in a Bash for loop, `do` must be preceded by a semicolon or placed on a new line (for service in "\${services[@]}; do). The script above uses the correct form.

Make it executable and run it:

A terminal window with a black background and green text. The prompt is 'manne@DESKTOP-QAJOUBH:~\$'. The first command is 'chmod +x optimize_services.sh'. The second command is './optimize_services.sh'. The output shows 'Starting service optimization...' followed by 'Disabling bluetooth ...'.

```
manne@DESKTOP-QAJOUBH:~$ chmod +x optimize_services.sh
manne@DESKTOP-QAJOUBH:~$ ./optimize_services.sh
Starting service optimization...
Disabling bluetooth ...
```

Figure 5: chmod +x and running the script (the script prints each service it disables)

After the script completes, the system is rebooted so the changes take effect at the next startup:
`sudo reboot.`

7. Measurement After Optimization

```
manne@DESKTOP-QAJOUBH:~$ systemd-analyze
Startup finished in 7.527s (userspace)
graphical.target reached after 7.381s in userspace.
manne@DESKTOP-QAJOUBH:~$ |
```

Figure 6: systemd-analyze after optimization - startup finished in 7.527 s

```
manne@DESKTOP-QAJOUBH:~$ systemd-analyze blame | head -10
3.007s landscape-client.service
1.685s wsl-pro.service
1.188s dev-sdd.device
1.125s snapd.seeded.service
911ms snapd.service
334ms systemd-udev-trigger.service
302ms systemd-journal-flush.service
300ms systemd-resolved.service
285ms rsyslog.service
255ms keyboard-setup.service
manne@DESKTOP-QAJOUBH:~$ |
```

Figure 7: ten slowest units after optimization

Rank	Service / Unit	Time taken
1	landscape-client.service	3.007 s
2	wsl-pro.service	1.685 s
3	dev-sdd.device	1.188 s
4	snapd.seeded.service	1.125 s
5	snapd.service	911 ms
6	systemd-udev-trigger.service	334 ms
7	systemd-journal-flush.service	302 ms
8	systemd-resolved.service	300 ms
9	rsyslog.service	285 ms
10	keyboard-setup.service	255 ms

8. Results and Comparison

Parameter	Before optimization	After optimization	Change
Startup finished (userspace)	22.287 s	7.527 s	-14.760 s (-66.2%)
graphical.target reached	21.861 s	7.381 s	-14.480 s (-66.2%)
Slowest unit	dev-sdd.device, 4.242 s	landscape-client.service, 3.007 s	-
Units above 1 s (top-10 list)	9	4	-

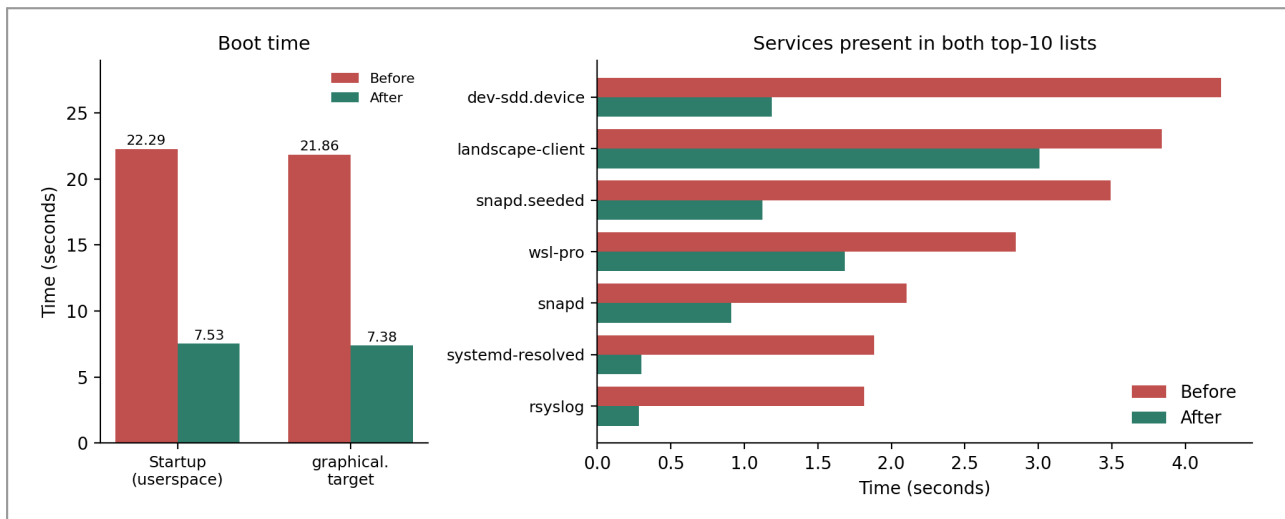


Figure 8: before vs after - total boot time and services present in both top-10 lists

Units that appear in both lists became noticeably faster:

Unit	Before	After
dev-sdd.device	4.242 s	1.188 s
landscape-client.service	3.841 s	3.007 s
snapd.seeded.service	3.493 s	1.125 s
wsl-pro.service	2.846 s	1.685 s
snapd.service	2.103 s	911 ms
systemd-resolved.service	1.884 s	300 ms
rsyslog.service	1.817 s	285 ms

9. Discussion

The measured userspace startup time fell by about two thirds after the optimization script was applied and the system was rebooted. Fewer units were active during startup, and the number of units taking more than one second in the top-10 list dropped from nine to four.

How to interpret the result. Bluetooth, CUPS and Avahi did not appear in the baseline top-10 list, so their direct contribution to the 22.287 s figure was small. The largest time savings were seen in other units (dev-sdd.device, snapd.seeded, snapd, wsl-pro), which suggests that part of the improvement

also comes from normal run-to-run variation - for example, first start of the WSL environment versus a later start with warm caches. For a rigorous result, repeat each measurement several times (for example 5 boots before and 5 after) and compare the averages. Also review the list with `systemd-analyze blame` and `systemd-analyze critical-chain` to decide which units are truly safe to disable or to defer, such as `landscape-client` and `snapd`, which remain the slowest units.

Scope. On WSL the firmware, bootloader and kernel stages are handled by Windows, so only the userspace phase could be measured here. On a real Embedded Linux board, bootloader tuning and kernel trimming can give further improvement.

10. Advantages, Limitations and Conclusion

Advantages

- Teaches the Embedded Linux boot flow and practical Linux commands
- Simple, repeatable method using only native `systemd` tools
- Reduces unnecessary service loading; can be extended to real hardware

Limitations

- Results depend on the hardware, distribution and environment
- Some services are required and cannot be disabled
- Incorrect optimization may affect system functionality
- Full embedded optimization needs board-specific knowledge (bootloader, kernel configuration)

Conclusion

The project shows a practical workflow for Linux boot optimization: measure, identify, optimize, reboot and compare. In the test run the userspace startup time was reduced from 22.287 s to 7.527 s. The same method can be applied to Ubuntu systems and, with kernel and bootloader tuning, to real embedded boards.

Future Scope

- Optimize U-Boot bootloader delay
- Build a custom lightweight kernel (`make menuconfig`, `make -j4`)
- Reduce root filesystem size and use BusyBox for smaller systems
- Optimize application startup and profile with `perf` and `ftrace`
- Apply the same method on Raspberry Pi or other embedded boards

References

- Linux Kernel Documentation - <https://www.kernel.org/doc/>
- Ubuntu Documentation - <https://ubuntu.com/>
- systemd Documentation - <https://www.freedesktop.org/wiki/Software/systemd/>
- U-Boot Documentation - <https://docs.u-boot.org/>